

Final Project

Generative Adversarial Networks

Implementation of Discriminator Rejection Sampling

Course: Deep Learning 2
Institution: Paris Dauphine-PSL

Sofia Gryn
Eric Chen

December 2025

1 Introduction

Generative Adversarial Networks (GANs) are widely used for sample generation from complex data distributions. Rather than modifying the training process, this project explores a post-processing approach to improve the quality of generated samples. We focus exclusively on Discriminator Rejection Sampling (DRS), detailing its intuition, implementation, and empirical behavior on MNIST dataset.

1.1 Preliminaries

In the GAN framework, the optimal discriminator is given by

$$D^*(x) = \frac{p_d(x)}{p_d(x) + p_g(x)},$$

where $p_d(x)$ denotes the real data distribution and $p_g(x)$ the generator distribution. At equilibrium, the generator reproduces the data distribution, i.e. $p_g(x) = p_d(x)$, which implies

$$D^*(x) = \frac{1}{2}.$$

This means that the discriminator cannot distinguish between real and generated samples. However, individual generated samples may still differ in how closely they resemble real data. This information provides a relative measure of how closely a sample resembles the real data distribution. In the next section, we show how this information can be exploited through Discriminator Rejection Sampling.

2 Discriminator Rejection Sampling

Discriminator Rejection Sampling (DRS) is a post-processing technique that uses a trained discriminator to probabilistically accept or reject samples generated by a fixed generator. We will first briefly explain the intuition behind the method, and then show its result on MNIST dataframe.

2.1 Intuition behind Rejection sampling

We rely on a simple one-dimensional example introduced in *Discriminator Rejection Sampling* (Azadi et al., 2019).

Rejection sampling is a method used to sample from a target distribution $p_d(x)$ by relying on an easier proposal distribution $p_g(x)$. The idea is to generate samples from the proposal distribution and then randomly reject some of them so that the remaining samples follow the target distribution.

To apply rejection sampling, we introduce a constant $M > 0$ such that

$$p_d(x) \leq Mp_g(x) \quad \forall x.$$

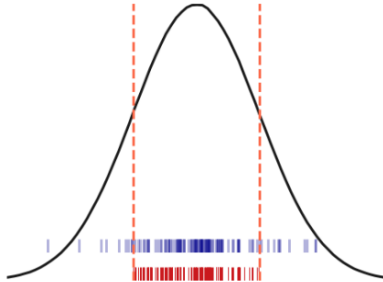


Figure 1: One-dimensional example: The real data distribution $p_d(x)$ follows a Gaussian distribution and is shown by the black curve. The generator distribution $p_g(x)$ is uniform over a fixed interval. Naive thresholding (red) removes the tails of the distribution, while rejection sampling (blue) preserves the shape of the target Gaussian.

The scaled distribution $Mp_g(x)$ acts as an envelope that fully contains the target distribution. This condition guarantees that the acceptance probability

$$0 \leq \frac{p_d(x)}{Mp_g(x)} \leq 1,$$

and can therefore be interpreted as a valid probability.

Sampling proceeds by first drawing a value x from the proposal distribution,

$$x \sim p_g(x).$$

For this fixed x , a vertical value is then sampled uniformly between 0 and $Mp_g(x)$. If this value lies below the target curve, meaning it is less than $p_d(x)$, the sample x is accepted; otherwise, it is rejected. In practice, this procedure is implemented by drawing a uniform random number

$$\psi \sim \mathcal{U}(0, 1)$$

and accepting the sample if

$$\psi < \frac{p_d(x)}{Mp_g(x)}.$$

This random acceptance step is essential because rejection sampling is a thinning process rather than a hard cutoff (corresponding to the red samples in Figure 1). It removes only the excess mass of the envelope while preserving the shape of the target distribution. As a result, the distribution of accepted samples (corresponding to the blue samples in Figure 1) matches exactly the target distribution $p_d(x)$.

2.2 Rejection Sampling in GAN setting

In the GAN setting, neither $p_d(x)$ nor $p_g(x)$ is known explicitly. However, as shown in [1], the discriminator can be used to estimate the density ratio between

the real and generated distributions. In particular, Equation (5) of [1] states that

$$\frac{p_d(x)}{p_g(x)} = \exp(\tilde{D}(x)),$$

where $\tilde{D}(x)$ denotes the discriminator logit.

From equation Equation (4) of [1] we derive the relationship between the discriminator output and its logit:

$$\tilde{D}(x) = \log\left(\frac{D(x)}{1 - D(x)}\right).$$

The acceptance probability can therefore be rewritten as

$$\frac{p_d(x)}{Mp_g(x)} = \exp(\tilde{D}(x) - \tilde{D}_M),$$

To avoid extremely low acceptance probabilities in high-dimensional settings, Azadi et al. [1] propose a stabilized acceptance function. Instead of directly using $\exp(\tilde{D}(x) - \tilde{D}_M)$, the final acceptance function is defined as

$$F(x) = \tilde{D}^*(x) - \tilde{D}_M^* - \log(1 - \exp(\tilde{D}^*(x) - \tilde{D}_M^* - \epsilon)) - \gamma,$$

and the acceptance probability is given by

$$\alpha(x) = \sigma(F(x)).$$

Here, γ is a hyperparameter controlling the overall acceptance rate, and ϵ is a small constant introduced for numerical stability (clamping in practice).

2.3 Implemented Algorithm

Algorithm 1 Discriminator Rejection Sampling (DRS) Implementation

Require: Generator G , discriminator D , required samples n , batch size B , stability ϵ , hyperparameter γ

- 1: Set G, D to evaluation mode; $M \leftarrow -\infty$; $\mathcal{A} \leftarrow \emptyset$; $k \leftarrow 0$
- 2: **while** $k < n$ **do**
- 3: Sample latent $z \sim \mathcal{N}(0, I)$ and generate $x \leftarrow G(z)$ $\triangleright |x| = B$
- 4: Compute discriminator outputs $D(x)$ and clamp: $D(x) \leftarrow \text{clamp}(D(x), \epsilon, 1 - \epsilon)$
- 5: Compute logits (Eq. 4): $\tilde{D}(x) \leftarrow \log D(x) - \log(1 - D(x))$
- 6: Update running maximum: $M \leftarrow \max(M, \max \tilde{D}(x))$
- 7: $\Delta \leftarrow \tilde{D}(x) - M - \epsilon$; $\Delta \leftarrow \min(\Delta, -\epsilon)$ $\triangleright$ ensures $\exp(\Delta) < 1$
- 8: Compute stabilized score (Azadi et al.): $F(x) \leftarrow \tilde{D}(x) - M - \log(1 - \exp(\Delta)) - \gamma$
- 9: Acceptance probability: $\alpha(x) \leftarrow \sigma(F(x))$
- 10: Sample $u \sim \mathcal{U}(0, 1)$; keep mask $m \leftarrow I[u < \alpha(x)]$
- 11: Append accepted samples $\mathcal{A} \leftarrow \mathcal{A} \cup x[m]$; update count $k \leftarrow k + |x[m]|$
- 12: **end while**
- 13: Shuffle $\mathcal{A}$ and return n samples uniformly at random from $\mathcal{A}$

Table 1: Evaluation results for different DRS configurations.

Note: The baseline generator is shown in **bold**, while the best-performing configuration is highlighted in gray.

FID	Precision	Recall	Acceptance Rate(last)%	γ
34.2	0.49	0.20	—	—
50.65	0.67	0.46	0.04	0
48.59	0.66	0.43	5	-3
48.34	0.66	0.44	12	-3.5
35.68	0.5	0.2	30	-5
33.87	0.49	0.22	30	-5

3 Results

3.1 Acceptance Rate, γ , and Precision–Recall Relationship

Table 1 illustrates the effect of the hyperparameter γ on the acceptance rate in Discriminator Rejection Sampling. As γ decreases, the acceptance rate increases substantially: when $\gamma = 0$, the acceptance rate remains very low (0.04%), whereas for $\gamma = -3.5$ and $\gamma = -5$, it rises to 12% and 30%, respectively. This confirms that γ directly controls the strictness of the rejection criterion imposed by the discriminator.

As the acceptance rate increases, both precision and recall decrease. The reduction in precision is expected, as higher acceptance rates allow lower-quality MNIST samples to be retained, reducing overall sample realism. The decrease in recall is less obvious.

Recall measures how well the generator covers the support of the real data distribution, here corresponding to the diversity of MNIST digit classes and writing styles. Increasing the acceptance rate does not modify the generator itself but instead relaxes the rejection criterion. Since discriminator scores are uneven across digit classes, this relaxation mainly increases the number of accepted samples from already well-represented digits, while poorly generated digits remain underrepresented. As a result, mode imbalance across MNIST classes is amplified, leading to a reduction in recall.

3.2 Impact of Discriminator Rejection Sampling on Precision and Recall

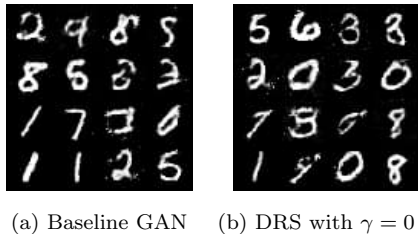


Figure 2. Comparison of MNIST samples generated by the baseline model and by Discriminator Rejection Sampling (DRS). The baseline GAN exhibits lower sample quality and limited diversity, consistent with its lower precision (0.49) and recall (0.20). In contrast, DRS with $\gamma = 0$ produces sharper digits and improved coverage, reflected by higher precision (0.67) and recall (0.46).

3.3 FID and Precision–Recall Relationship

We observe a consistent pattern: when both precision and recall decrease, FID increases. Conversely, configurations with higher precision and recall tend to exhibit worse FID scores, reflecting the different aspects captured by these metrics. Nevertheless, compared to the baseline generator, we reduce the initial FID by improving recall, indicating an overall improvement in model performance.

3.4 Computational Cost and Acceptance Rate

Discriminator Rejection Sampling (DRS) introduces an additional computational overhead compared to a standard GAN, since instead of accepting all generated samples, each sample is evaluated by the discriminator and accepted only with a probability proportional to its estimated realism.

As a consequence, multiple generated samples may be rejected before obtaining a valid one, which increases the number of forward passes through both the generator and the discriminator. For a target of n accepted samples and an average acceptance rate α , the expected number of generated samples scales as n/α . Therefore, when the acceptance rate is low, the effective sampling time increases significantly.

4 To Go Further

Discriminator Rejection Sampling can be further improved by introducing a sampling budget through *Optimal Budgeted Rejection Sampling* (OBRS). This variant addresses one of the main limitations of DRS, namely its sensitivity to the estimation of the upper bound M and to the hyperparameter γ .

OBRS enforces a fixed budget K of generated samples per accepted sample, which results in an expected acceptance probability of $1/K$. By explicitly controlling the number of rejection attempts, the acceptance rate becomes more stable and predictable. Consequently, the sampling process is less sensitive to inaccurate upper-bound estimates and exhibits improved robustness compared to standard DRS.

References

- [1] Samaneh Azadi, Catherine Olsson, Trevor Darrell, Ian Goodfellow, and Augustus Odena. Discriminator rejection sampling. In *International Conference on Learning Representations (ICLR)*, 2019.